Yash Honda Backend Decision Guide

Private Study Reference

1) What MVP Means

> **MVP (Minimum Viable Product)** is the smallest complete version of the system that delivers real business value and can be used in operations.

For this project, MVP means the system can:

- Create and calculate quotations correctly
- Save and retrieve quote records from SQL
- Support multi-user usage during normal operations
- Provide printable/PDF quote output
- Run securely with role-based access

2) Why This System Exists

The current process is spreadsheet-heavy and manual. That creates slower execution, inconsistency risk, and coordination overhead for the sales team. The new platform centralizes data and automates calculations so quoting is faster and more reliable.

3) Why REST API with Versioning

- Simple and reliable integration for frontend and backend teams
- JSON payloads are easy to test and debug
- Versioning (`/api/v1`) protects clients from breaking changes

Meeting explanation: "A versioned API contract keeps implementation aligned and stable over time."

4) Why SQL Is the Right Fit

- Quote data is relational and structured (customers, quotes, line items, users)
- Transactions improve data safety under concurrent activity
- Constraints and indexes improve correctness and performance
- Reporting and analytics are easier with structured schema

Meeting explanation: "SQL gives integrity, traceability, and operational reporting confidence."

5) Why Role-Based Access (RBAC)

Different users need different permissions:

- Sales users: create/manage quote workflows
- Managers/Admin: broader visibility and control
- Customer access: restricted, limited scope

RBAC enforces least privilege, reduces security risk, and keeps operational behavior clean.

6) How Shared Daily Quotes Should Work

- Active daily quotes remain visible on a shared sales workspace
- Point-of-sale categories are color-coded for faster recognition
- When concurrent quotes start, rows appear sequentially in the live list
- Backend is source of truth; frontend reflects current state consistently

This directly replaces spreadsheet coordination with a centralized, live operational model.

7) Why Non-Production and Production Separation Matters

- Non-production is used for development, integration, and testing
- Production is the stable live environment for dealership staff
- Changes are validated in non-production before promotion
- This keeps live operations available while enhancements continue

8) Why NFRs Need Early Definition

Functional requirements describe *what* the system does. Non-functional requirements describe *how well* it does it, including speed, reliability, security, and scalability.

Early NFR targets prevent technical debt and ensure the build supports real operational usage.

9) How Data Migration and Existing Metrics Integration Works

1. Catalog all existing data sources (Excel/legacy databases/files)
2. Map each current field to the new SQL schema
3. Define transformation rules (formats, enums, required fields)
4. Run staged migration in non-production
5. Validate totals, row counts, and sample records with stakeholders
6. Execute controlled cutover with monitoring

This process ensures legacy metrics are preserved accurately and remain useful after migration.

10) Access Management Basics

- Application access via role assignments (RBAC)
- Environment access via cloud IAM groups/roles
- Database access under least-privilege rules
- Audit logging on sensitive operations and admin actions

11) Questions You Should Collect in Meetings

- What are launch-critical features vs later enhancements?
- Which pricing rules are fixed vs likely to change?
- What historical data must be migrated and validated?
- Which metrics are mandatory for management reporting?
- Who approves production deployments?
- What is the go-live training and support plan?

12) Short Confidence Lines for Discussion

- "The architecture is designed for operational continuity and controlled change."
- "Backend and frontend are aligned through one versioned API contract."
- "Data integrity and concurrency are handled at both API and database layers."
- "Environment separation protects production while allowing rapid iteration."